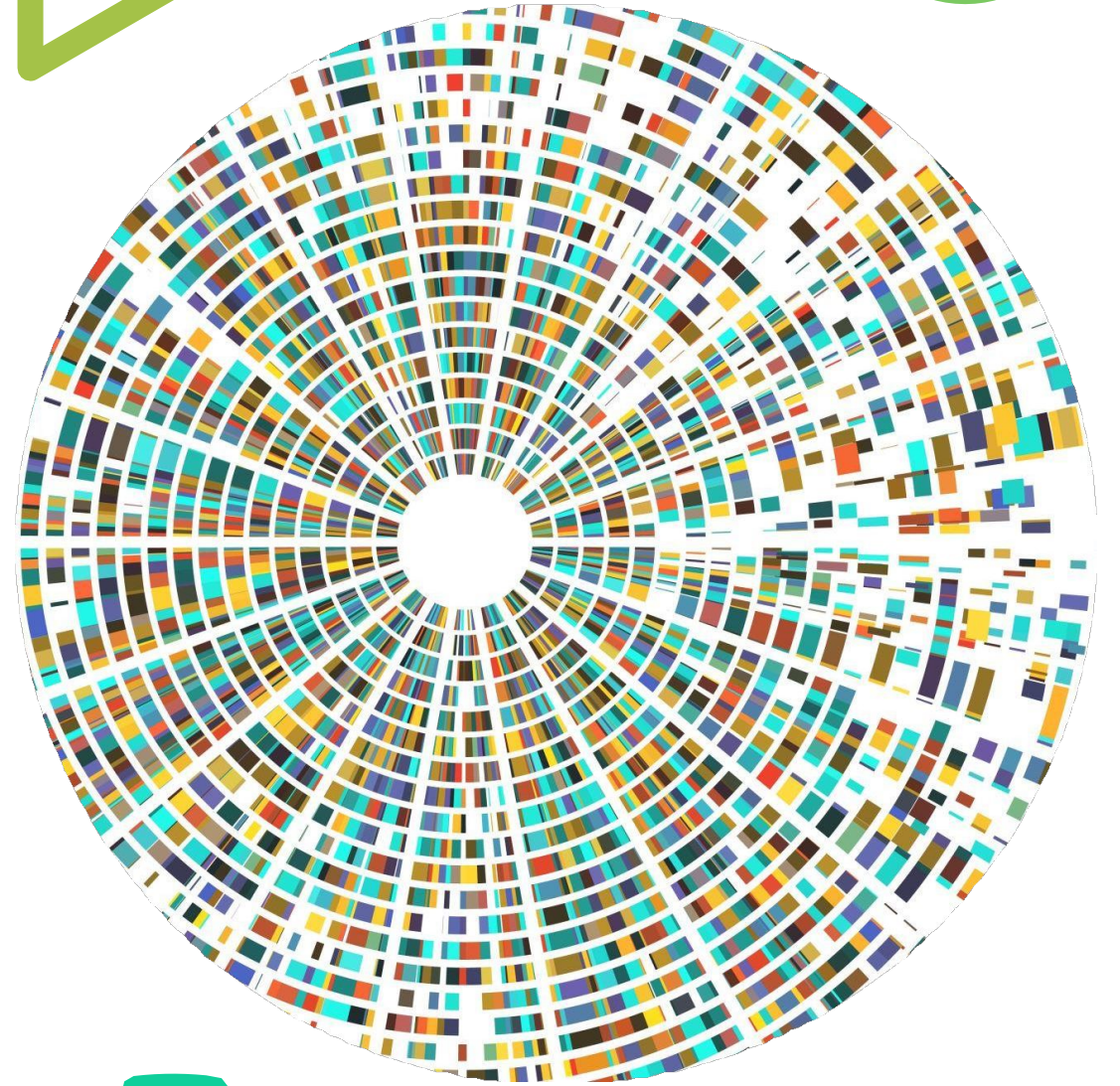




UT-6

Estructuras Dinámicas de Datos

1º DAM/DAW
(2024-2025)

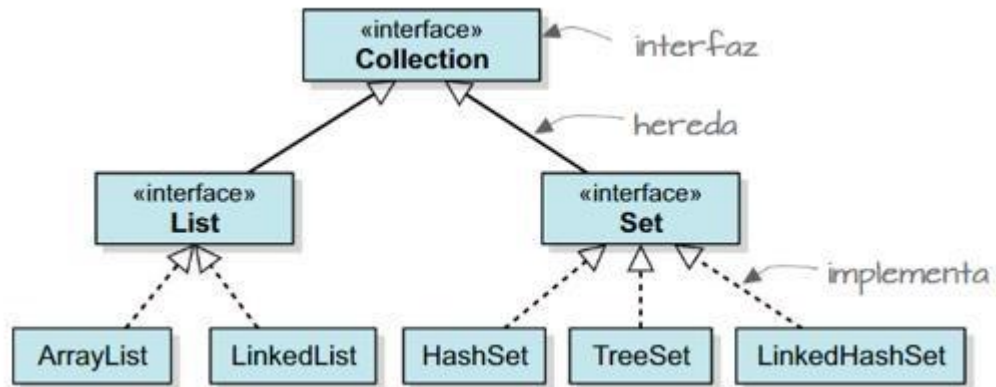


1 – Estructuras Dinámicas

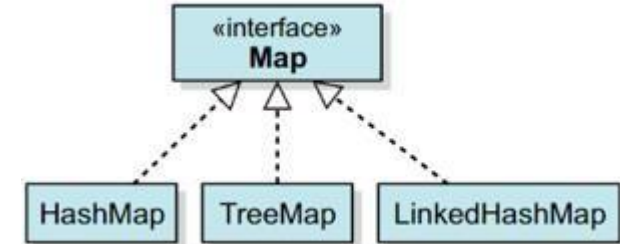
En Java, las estructuras de datos dinámicas permiten almacenar listas o colecciones de datos de un tipo determinado. Además, proporcionan una API muy completa de operaciones para poder realizar sobre toda la colección. Se pueden considerar como una evolución de los arrays ya que éstos son estáticos y tienen una operativa más básica.

Además, los diferentes tipos de colecciones permiten afinar el comportamiento de cada una de ellas de forma que podremos elegir la que más nos convenga para cada caso. Existen, por ejemplo, colecciones de datos en las que los elementos están automáticamente ordenados en todo momento (TreeSet) u otras que no admiten elementos repetidos (HashSet). Otras estructuras mantienen el orden de los elementos (LinkedList o ArrayList) mientras que en otras esa información no es relevante (HashMap)

Colecciones



Mapas o Diccionarios



1.1 – Estructuras Dinámicas (Características)

Al emplear colecciones, es importante tener en cuenta:

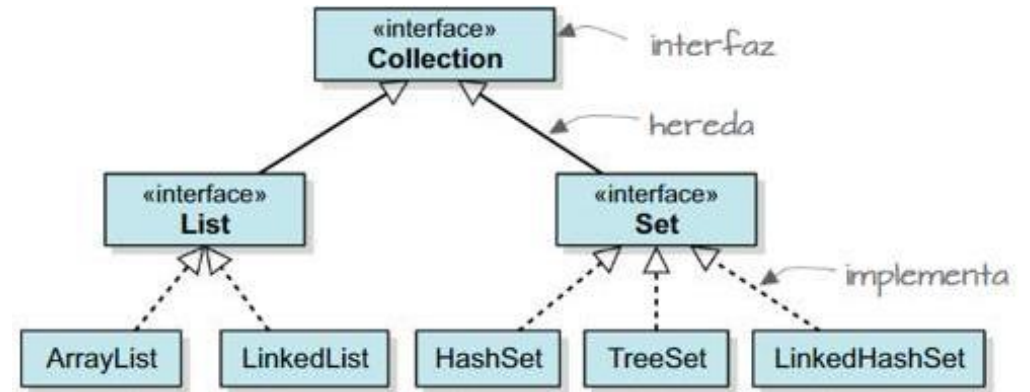
- Estudiar bien qué tipo de colección interesa más para cada caso (no usar siempre ArrayList porque sí)
- Implementar/Extender los métodos de la clase Object en las clases que diseñamos:
 - hashCode()
 - equals()
 - toString()
- Aprender a utilizar algunos interfaces interesantes para usar con colecciones:
 - Comparator
 - Comparable

Colección	Posicionamiento	Ordenación	Duplicados
TreeSet	Ninguno	Ascendente	NO
HashSet	Ninguno	Ninguna	NO
LinkedHashSet	Ninguno	Orden de Inserción	NO
ArrayList	Índice	Ninguna	SI
LinkedList	Índice	Orden de Inserción	SI
HashMap	Clave-Valor	Ninguna	Clave Única
TreeMap	Clave-Valor	Ascendente	Clave Única
LinkedHashMap	Clave-Valor	Orden de Inserción	Clave Única

2 - Interfaz Collection

Es la interface padre de todas las colecciones Java. Define una serie de métodos que cualquier colección tiene que implementar:

```
boolean add(E e)
boolean addAll(Collection<E> c)
void clear()
boolean contains(E e)
boolean isEmpty()
Iterator<E> iterator()
boolean remove(E e)
boolean removeAll(Collection<E> c)
int size()
Object[] toArray()
<T> T[] toArray(T[] a)
```



```
boolean hasNext();
E next ();
Void remove (Object o)
```


3 – Interfaz Set (Conjuntos)

La interfaz “Set”, por definición, **no acepta elementos duplicados**. No define métodos nuevos, solamente implementa Collection e impide que se repitan elementos.

HashSet

```
Set<Integer> conjunto = new HashSet<>();
```

- Rendimiento constante para las operaciones más habituales
- No garantiza el orden de los elementos de la colección

LinkedHashSet

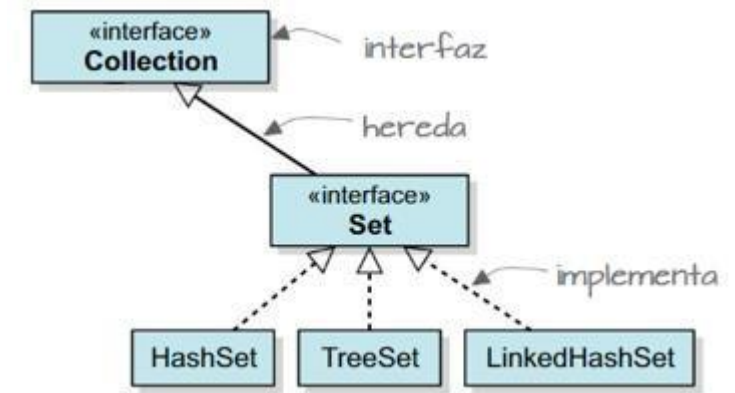
```
Set<Integer> conjunto = new LinkedHashSet<>();
```

- Mantiene una lista doblemente enlazada por lo que puede obtenerse un iterador con el orden en que los elementos fueron añadidos a la colección

TreeSet

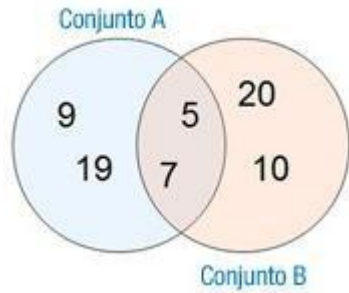
```
Set<Integer> conjunto = new TreeSet<>();
```

- Garantiza el orden de los elementos
- Sus elementos se mantienen siempre ordenados de forma natural o por un Comparator que se proporcione en su constructor.
- Su rendimiento es el menor de todos los Sets

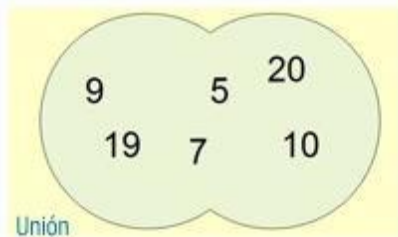


3.1 – Interfaz Set (Operaciones básicas)

Los conjuntos son útiles para implementar operaciones básicas de **Unión**, **Diferencia** e **Intersección**.

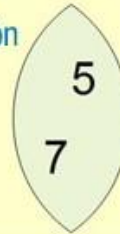


```
Set<Integer> conjuntoA = new HashSet<>(Arrays.asList(new Integer[]{9,19,5,7}));  
Set<Integer> conjuntoB = new HashSet<>(Arrays.asList(new Integer[]{20,10,5,7}));
```

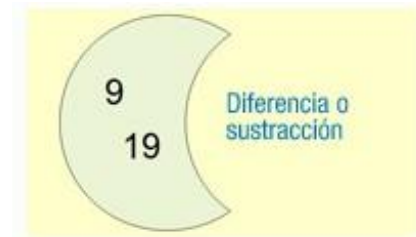


```
conjuntoA.addAll(conjuntoB); //UNIÓN A∪B  
conjuntoB.addAll(conjuntoA); //UNIÓN B∪A
```

Intersección



```
conjuntoA.retainAll(conjuntoB); // INTERSECCIÓN A∩B  
conjuntoB.retainAll(conjuntoA); // INTERSECCIÓN B∩A
```



```
conjuntoA.removeAll(conjuntoB); //DIFERENCIA A-B  
conjuntoB.removeAll(conjuntoA); //DIFERENCIA B-A
```

4 – Interfaz List (Listas)

La interfaz “**List**” es la interfaz padre de todas las colecciones tipo Lista. Define una serie de método que cualquier colección tiene que implementar:

- `E get(i)`
- `int indexOf(e)`
- `void sort(Comparator<? super E> c)`

ArrayList

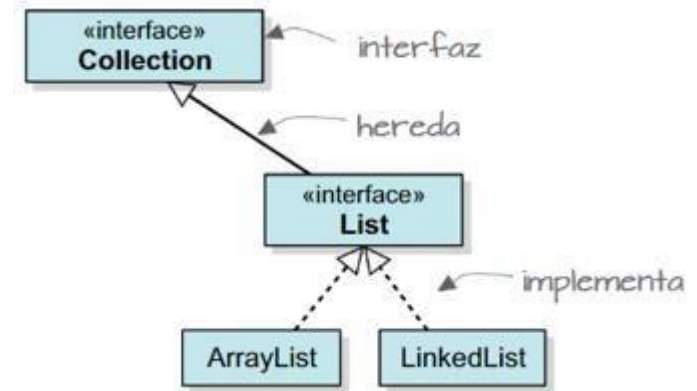
- La clase ArrayList permite definir arrays dinámicos. Son colecciones de datos dinámicas con el acceso y funcionamiento de un array, puesto que sólo es posible recuperar cada elemento a partir de la posición del mismo.

```
List<Object> lista = new ArrayList<>();
```

LinkedList

- La clase LinkedList es una lista doblemente enlazada, que permite almacenar una colección de objetos, obtenerlos, eliminarlos y algunas operaciones para acceder a los mismos. Acceder al primer y último elemento, entre otros, son operaciones muy directas en este tipo de colección.

```
List<Object> lista = new LinkedList<>();
```



4.1 – Interfaz List (ArrayList)

La clase **ArrayList** permite definir arrays dinámicos:

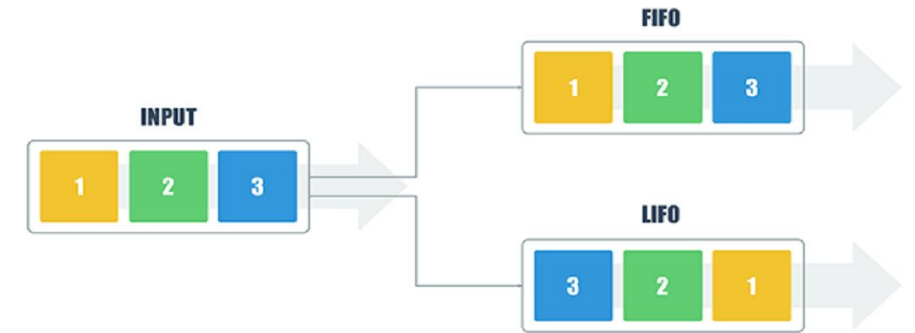
- Implementa la interface List.
- Es el tipo de colección más genérica.
- Define una colección de elementos sin ninguna característica especial.
- Los elementos se pueden obtener a partir de la posición donde fuera añadidos.

```
List<Object> lista = new ArrayList<>(); // Crear ArrayList.
List<Object> lista2 = new ArrayList<>();
lista.add(objeto1);                      // Añadir objeto.
lista2.add(objeto2);
lista.addAll(lista2);                    // Añadir otra colección al final.
Object objet1 = lista.get(0);            // Recuperar objeto de una lista.
Object objet2 = lista.remove(0);         // Eliminar objeto (puede devolver el objeto eliminado).
int longitud = lista.size();             // Consultar el tamaño de una lista.
Object [] listaArray = lista.toArray(); // Convierte una lista en un Array
lista.clear();                           // Borrar la lista.
```


4.2 – Interfaz List (Listas Enlazadas)

La clase **LinkedList** es una lista doblemente enlazada:

- Implementa la interface List
- Se utiliza para definir estructuras FIFO (First In First Out), también conocidas como colas.
- Define métodos que permiten, directamente, acceder/eliminar el primer elemento de una lista (el primer elemento de la cola).



La clase LinkedList implementa algunos métodos especiales.

```
LinkedList<Object> listaEnlazada = new LinkedList<>(); // Crear Lista Enlazada.  
listaEnlazada.addFirst(objeto1); // Añadir objeto al principio de la lista.  
listaEnlazada.addLast(objeto2); // Añadir objeto al final de la lista.  
Object primerObjeto = listaEnlazada.getFirst(); // Devuelve el primer objeto de la lista.  
Object ultimoObjeto = listaEnlazada.getLast(); // Devuelve el último objeto de la lista.  
Object ObjetoBorrado1 = listaEnlazada.removeFirst(); // Elimina el primer objeto de la lista.  
Object ObjetoBorrado2 = listaEnlazada.removeLast(); // Elimina el último objeto de la lista.
```

5 – Conversión entre colecciones

Una característica interesante de los conjuntos y de todas las colecciones en general es la posibilidad de crear unas a partir de otras, del mismo o distinto tipo, por medio de los constructores. Utilizando los constructores, es posible hacer conversiones entre todo tipo de colecciones. Se pueden crear listas pasando conjuntos a su constructor y viceversa. También se pueden añadir a una lista todos los elementos de un conjunto.

```
// Ejemplo ArrayList a Set para eliminar duplicados.  
List<Integer> listaNumeros= new ArrayList<>();  
listaNumeros.add(2);  
listaNumeros.add(3);  
listaNumeros.add(2); // Elemento repetido  
listaNumeros.add(3); // Elemento repetido  
Set<Integer> conjuntoNumeros = new HashSet<>(listaNumeros); // Sin repeticiones.
```

Cuando se trabaja con colecciones de distinto tipo, siempre que se usen constructores o métodos comunes a listas y conjuntos, es costumbre definir las variables de tipo **Collection** para permitir que la misma variable referencie a diferentes tipos de colección en caso de conversiones.

```
Collection<Integer> coleccion = new ArrayList<>();  
coleccion.add(5);  
coleccion = new LinkedHashSet<>(coleccion); //de lista a conjunto  
coleccion = new ArrayList<>(coleccion); //de conjunto a lista
```

6 – Clase Collections (Métodos Estáticos).

Además de los métodos aportados por las interfaces **Collection** , **List** y **Set**, la clase **Collections** (no confundirla con la interfaz **Collection**) reúne una serie de utilidades en forma de métodos estáticos que trabajan con tipos genéricos. En ellos, el primer parámetro de entrada es la colección sobre la que deseamos operar y comprende métodos de búsqueda, ordenación y manipulación de datos, entre otros. Casi todos ellos operan sobre listas, aunque algunos valen para cualquier colección:

- Métodos de Ordenación.

- `void sort (List<> lista);` // Ordena una lista
- `void sort (List<> lista, Comparator<> comparador);`

- Métodos de Búsqueda.

- `int binarySearch (List<> lista, Object objeto);` // Busca un objeto en la lista
- `int binarySearch (List<> lista, Object objeto, Comparator<> comparador);`

- Métodos de Manipulación.

- `void shuffle (List<> lista);` // Desordena una lista
- `void reverse (List<> lista);` // Invierte el orden de la lista
- `void swap (List<> lista, int indice1, int indice2);` // Intercambia las posiciones de dos objetos
- `void copy (List<> listaDestino, List<> listaOrigen);` // Copia una lista en otra
- `int frequency (Collection<> coleccion, Object objeto);`

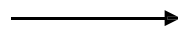
7 – Interfaz Comparable.

La interfaz **Comparable** se utiliza para definir el orden natural de los objetos de una clase.

La interfaz tiene un único método llamado **compareTo**, que debe ser implementado por la clase que desea ser comparable. La implementación de **compareTo** devuelve un número negativo si el objeto actual es menor que el objeto proporcionado, cero si son iguales y un número positivo si el objeto actual es mayor.

Podemos personalizar el criterio de ordenación de las colecciones sobrecargando el método **compareTo** en la clase base que queramos ordenar.

Ordenar Personas por edad.



```
public class Persona implements Comparable<Persona> {  
    private String nombre;  
    private int edad;  
  
    // Constructor y otros métodos  
  
    @Override  
    public int compareTo(Persona otraPersona) {  
        return this.edad - otraPersona.edad;  
    }  
}
```

8 – Interfaz Comparator.

La interfaz **Comparator** se utiliza para definir múltiples formas de comparar objetos de una clase sin modificar su implementación.

A diferencia de Comparable, donde la lógica de comparación está definida en la propia clase, Comparator permite definir comparadores externos.

La interfaz tiene un método llamado **compare** que toma dos objetos y devuelve un número negativo si el primer objeto es menor, cero si son iguales y un número positivo si el primer objeto es mayor.

Ordenar Personas por Nombre. →

```
public class comparadorPorNombre implements Comparator <Persona>{  
  
    @Override  
    public int compare(Persona persona1, Persona persona2) {  
        return persona1.getNombre().compareTo(persona2.getNombre());  
    }  
  
}
```

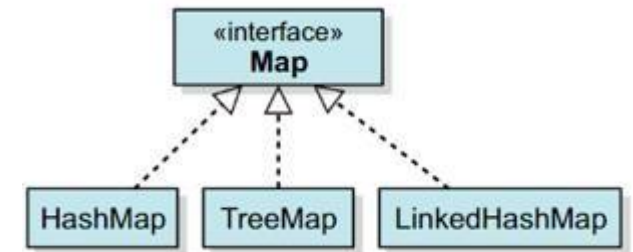
Se pueden emplear los métodos estáticos **comparing()**, **thenComparing()** y **reversed()** de la clase Comparator para crear comparadores.

```
// Ordenar la lista por edad y luego por nombre (en orden inverso)  
Comparator comparador= Comparator.comparing(Persona::getEdad)  
                                .thenComparing(Persona::getNombre)  
                                .reversed();
```


9 – Interfaz Map (Mapas o Diccionarios).

La interfaz **Map** es una colección de elementos que se encuentran mapeados por una clave (**parejas clave-valor (K,V)**). La clave asociada a cada elemento permite recuperarlo y no se permiten claves duplicadas (se sobrescribe el valor si la clave se vuelve a registrar).

Los valores pueden ser cualquier tipo de objeto.



Define una serie de métodos que cualquier colección tiene que implementar:

- `Void put(Key k, Value v)` // Coloca un par K-V en el mapa.
- `Value get(Key k)` // Devuelve el valor asociado a una clave.
- `void clear(Map mapa)` // Borra el contenido de un mapa.
- `boolean containsKey(Key k)` // Indica si existe una clave en el mapa.
- `boolean containsValue(Value v)` // Indica si existe un valor en el mapa.
- `boolean isEmpty(Map mapa)` // Indica si el mapa está vacío.
- `Set<K> keySet(Map mapa)` // Devuelve el conjunto de claves del mapa.
- `V remove(Key k)` // Elimina un elemento del mapa
- `V replace(Key k, Value nuevo)` // Reemplaza el valor asociado a una clave.
- `int size(Map mapa)` // Indica el tamaño del mapa.
- `Collection<V> values(Value v)` // Devuelve el conjunto de valores de un mapa.

9.1 – Interfaz Map (Mapas o Diccionarios).

La interfaz **Map** se implementa en tres clases principales

HashMap

- Es un mapa que proporciona rendimiento constante para las operaciones **put()** y **get()**.

```
Map<String, Persona> listadoPersonas = new HashMap<>();
```

LinkedHashMap

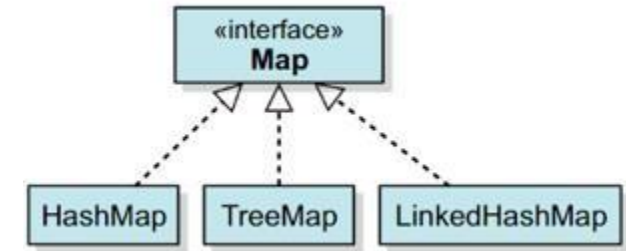
- Mantiene una lista doblemente enlazada
- Es posible obtener un iterador con el orden de inserción de las claves

```
Map<String, Persona> listadoPersonas = new LinkedHashMap<>();
```

TreeMap

- Es un mapa ordenado por el orden natural o Comparator que se indique en el constructor

```
Map<String, Persona> listadoPersonas = new TreeMap<>();
```



9.2 – Iteración sobre Mapas.

En un mapa podemos iterar sobre tres elementos.

- **Iteración sobre claves**

```
for (String clave: mapaPersonas.keySet()) {  
    System.out.print(clave + " ->");  
    System.out.println(mapaPersonas.get(clave));  
}
```

- **Iteración sobre valores**

```
for (Persona p: mapaPersonas.values()) {  
    // De este modo no se puede acceder a las claves  
    System.out.println(p);  
}
```

- **Iteración sobre entradas**

```
for (Map.Entry<String,Persona> entrada: mapaPersonas.entrySet()){  
    System.out.print(entrada.getKey()+" ->");  
    System.out.println(entrada.getValue());  
}
```